



华中科技大学计算机科学与技术学院 2023~2024 第一学期

“ 计算机系统基础 ” 考试试卷 (B 卷)

考试方式 闭卷 考试日期 2024-3-18 考试时长 150 分钟

专业班级 学 号 姓 名

题号	一	二	三	四	五	总分	核对人
分值	20	20	20	20	20	100	
得分							

分 数	
评卷人	

一、数据的存储表示和访问 (共 20 分)

1、数据的存储表示 (10 分)

```
void func1()
{
    int x = 10;
    int y[2] = { 20, 32 };
    char msg[6] = "abcde";
    int* p = &x;
    short z = *(short*)(msg + 1);
    int u = *(int*)(msg + 1);
}
```

设 x 的地址(&x) 为: 0xffffd508; y[0] 的地址为: 0xffffd50c; msg[0] 的地址为: 0xffffd51a;

p 的地址(&p) 为: 0xffffd514; z 的地址为: 0xffffd518; u 的地址为: 0xffffd520。

在函数 func1 执行结束返回前, 以字节形式观察内存内容。以 16 进制的形式填写程序用到的内存单元内容 (无关的字节中的内容不要填写, 打 XX)。

0xffffd508	___	___	___	___	___	___	___
0xffffd510	___	___	___	___	___	___	___
0xffffd518	___	___	___	___	___	___	___
0xffffd520	___	___	___	___	___	___	___

2、调试一个 C 程序时, 在函数的反汇编中看到源程序语句与对应的反汇编语句如下。根据

观察到的信息填空。（10 分）

```
int a[2][5];
```

```
int i = 1;
```

```
movl $1, -0x2c(%ebp)
```

i 的地址为 0x0019feb8, ebp = 0x _____

```
int j = 3;
```

```
movl $3, -0x30(%ebp)    &j = 0x_____
```

```
a[i][j] = 10;
```

```
imul $0x14, -0x2c(%ebp), %eax
```

执行后, eax = _____

eax 中值的含义是_____

```
lea -0x28(%ebp, %eax, 1), %ecx
```

ecx 中存放的值的含义是_____

```
mov -0x30(%ebp), %edx
```

```
movl $0xa, (%ecx, %edx, 4)
```

指令中的 edx 乘 4 的原因是_____

a[0][0] 的地址表达形式是 _____;

```
global[i] = 18;    global 为一个全局整型数组
```

```
mov -0x2c(%ebp), %eax
```

```
movl $0x12, 0x417120(, %eax, 4)
```

global 数组的起始地址是_____,

```
int* p = &global[1];
```

```
mov $4, %eax
```

```
shl $0, %eax
```

```
add $0x417120, %eax
```

```
mov %eax, -0x34(%ebp)
```

变量 p 中的值是 0x_____

```
*p = 19;
```

```
mov -0x34(%ebp), %eax
```

```
movl $0x13, (%eax)
```

第二条汇编语句访存的寻址方式是_____

解答内容不得超过装订线

分 数	
评卷人	

二、程序运行的基本过程（共 20 分）

1、在对某程序进行调试时，看到如下一段信息（10 分，每空 1 分）：

① 设当前 `eip` 为 `0x00411650`，该处指令为“`83 7D FC 00 cml $0,-4(%ebp)`”，在取出该指令并进行译码后，`eip` = _____；

② 在 `0x00411654` 处，有指令“`7E 08 jle 0041165E`”，综合前一条指令，就是在_____条件下，将 `0x0041165E` $\rightarrow$ `eip`，计算出该地址值的方法是 _____；

③ 在 `0x0041165C` 处，有 `jmp` 指令“`EB 08`”，该指令对应的反汇编语句是：_____；执行该指令时，会_____（有、无）条件跳转到 `0x00411666` 处；

④ 在 `0x00411666` 处，有一子程序直接调用指令“`E8 65 FC FF FF callq 004112D0`”。由此推断子程序的入口地址是 _____；指令中 `FFFFFC65` 的含义是_____；执行该 `call` 指令时，CPU 会将 `0x_____` 压入堆栈中；

⑤ 在 `0x00411672` 处，有指令“`FF 55 F4 callq -0xc(%ebp)`”，得到子程序入口地址的方法是_____；

⑥ 执行 `RET` 指令时，CPU 会_____ $\rightarrow$ `eip`。

2、函数调用（10 分）

在Linux环境下，对一个C语言程序进行编译、链接、调试运行，程序片段如下。

```
int fmax(int a, int b) {
    004115A5 push    %ebp
    004115A6 mov     %esp,%ebp
    004115A8 sub     $0x10,%esp
    int result;
    if (a > b)
        004115A9 mov     0x8(%ebx), %eax
        004115AC cmp     0xc(%ebp), %eax
        004115AF jle     fmax+19h (04115B9h)
        result = a;
        004115B1 mov     0x8(%ebp), %eax
        004115B4 mov     %eax, -0x4(%ebp)
        004115B7 jmp     fmax+1Fh (04115BFh)
    else result = b;
        004115B9 mov     0xc(%ebp), %eax
        004115BC mov     %eax, -0x4(%ebp)
    return result;
```

```

004115BF  mov     -0x4(%ebp), %eax
}

004115C2  mov     %ebp, %esp
004115C4  pop     %ebp
004115C5  ret

void main( ) { .....
    int  x = fmax(10, -10);
    00413863  push     $0xffffffff6h
    00413865  push     $0xa
    00413867  call     004115A5
    0041386C  add     $8, %esp
    0041386F  mov     %eax, -0x4(%ebp)
    .....
}

```

设执行 main 函数中的 call 004115A5 之前, esp= 0xffffd500; ebp= 0xffffd510.

填写刚进入函数 fmax 时（执行 push %ebp 之前），堆栈中相关单元的内容。

函数参数 a 的地址（即&a）是 0x_____。

局部变量 result 的地址是 0x_____。

	0xffffd4f0
	0xffffd4f4
	0xffffd4f8
	0xffffd4fc
XXXXXXXXXX	0xffffd500

分 数	
评卷人	

三、C 语句的转换 （共 20 分）

1、阅读下面的程序，回答问题 （10 分，每小题 2 分）。

```

void func ()
{
    unsigned short  us_x = 0x8000;
    unsigned int    ui_x;
    short  s_x = -0x8000;
    int    i_x;
    int    u = 0, v = 0;           // ①
    ui_x = us_x;
    i_x = s_x;                     // ②
    if (ui_x > 0) u = -1;
    if (i_x > 0) v = -1;           // ③
    ui_x = ui_x >> 1;
    i_x = i_x >> 1;                // ④
    ui_x = ~ui_x;
    i_x = -i_x;                    // ⑤
}

```

(1) 执行完 ① 处语句后，

us_x = 0x _____ s_x = 0x _____ (2个字节的16进制数)

(2) 执行完 ② 处语句后,

ui_x = 0x _____ i_x = 0x _____ (4个字节的16进制数)

ui_x = us_x; 对应的机器指令: _____ us_x, %eax 、 mov %eax, ui_x

i_x = s_x; 对应的机器指令: _____ s_x, %eax 、 mov %eax, i_x

(3) 执行完 ③ 处语句后,

u = _____ v = _____

if (ui_x > 0) u = -1; 对应的机器指令: cmp \$0, ui_x, _____ process2、
mov \$-1, u, _____ process2:...

if (i_x > 0) v = -1; 对应的机器指令: cmp \$0, i_x, _____ process3、
mov \$-1, v, _____ process3:...

(4) 执行完 ④ 处语句后,

ui_x = 0x _____ i_x = 0x _____ (4个字节的16进制

数)

ui_x = ui_x >> 1; 对应的机器指令: _____ \$1, ui_x

i_x = i_x >> 1; 对应的机器指令: _____ \$1, i_x

(5) 执行完 ⑤ 处语句后,

ui_x = 0x _____ i_x = 0x _____ (4个字节的16进制数)

ui_x = ~ui_x; 对应的机器指令: _____ ui_x

i_x = -i_x; 对应的机器指令: _____ i_x

2、程序优化 (10分)。

设有如下一段C语言程序, 请写出完成相同功能、执行速度尽可能快的汇编语言程序代码片段, 并指出优化中利用了CPU中的哪些特性。

```
int a[5], i, sum=0;
```

```
.....
```

```
for (i = 0; i < 5; i++) sum += a[i];
```

分 数	
评卷人	

四、程序阅读与理解（20 分）

1、阅读下面的程序，回答问题（10 分）。

```
.section .data
    array: .long -1, -2, 11, -9, 5, 100
    length = (. -array)/4          # length 为array中元数的个数, = 6
    format: .ascii "%d\n\0"

.section .text
.global _start
_start:
    mov $0, %eax
    mov $length, %ecx
    mov $0, %esi                  # ①
lp_1:
    cmpl $10, array(, %esi, 4)
    jg lp_2                      # ②
    inc %eax
lp_2:
    inc %esi
    sub $1, %ecx                 # ③
    jne lp_1
    push %eax
    push $format
    call printf
    mov $1, %eax                # 程序正常退出
    mov $0, %ebx
    int $0x80
```

1) 上述程序的功能是什么？运行后，屏幕上显示的是什么？（2 分）

2) 若标号 lp_1 写到 ①处语句前，程序运行的结果是什么？为什么？（3 分）

3) 若将 ② 处的语句改为 “ja lp_2”，程序运行的结果是什么？（2 分）

4) 若漏写了 ③ 处的语句，程序运行会出现或什么现象？为什么？（3 分）

2. 程序填空（10 分，每空 1 分）

将一个以 0 结束的字符串中的所有的大写字母转换为对应的小写字母，并将转换后的字符串、以及修改的字母个数输出。

```
.....
.section .data
    format: .ascii "%s %d\n\0"
    buffer: .ascii "Computer System Foundation \n\_\\_"
.section .text
.global _start
_start:
    mov _____, %ebx    # ebx 存放 buffer 数组元素的下标
    mov _____, %ecx    # ecx 存放修改字母的个数
loop_begin:
    mov  buffer(%ebx), %al
    cmp  $0, %al
    _____
    cmp  $'A', %al
    _____
    cmp  $'Z', %al
    _____
    add  _____, %al
    mov  %al, _____
    _inc _____
next_char:
    _____
    jmp  loop_begin
loop_over:
    push %ecx
    pushl $buffer
    pushl $format
    call printf
    mov  $1, %eax # 程序正常退出
    mov  $0, %ebx
    int  $0x80
```

分 数	
评卷人	

五、问答题（20 分）

- 1、可重定位目标文件中有哪些节？各节中主要有什么信息？什么是符号解析？链接的过程是什么？（10 分）
- 2、在一个程序的运行过程中（如正在执行一个二维数组求累加和），用户按了键盘上的某个键，计算机系统会做出哪些响应（即一系列的处理过程）？中断分哪几类？请举例说明各类中断在何种情况下产生。（10 分）